12 Fetch plans, strategies, and profiles

This chapter covers

- Working with lazy and eager loading
- Applying fetch plans, strategies, and profiles
- Optimizing SQL execution

In this chapter, we'll explore Hibernate's solution for the fundamental ORM problem of navigation, as introduced in section 1.2.5: the difference in how you access data in Java code and within a relational database. We'll demonstrate how to retrieve data from the database and how to optimize this loading.

Hibernate provides the following ways to get data out of the database and into memory:

- We can retrieve an entity instance by identifier. This is the most convenient method when the unique identifier value of an entity instance is known, such as `entityManager.find(Item.class, 123)`.
- We can navigate the entity graph, starting from an already-loaded entity instance, by accessing the associated instances through property accessor methods such as `someItem.getSeller().getAddress().getCity()`, and so on. Elements of mapped collections are also loaded on demand when we start iterating through a collection. Hibernate automatically loads nodes of the graph if the persistence context is still open. What data is loaded when we call accessors and iterate through collections, and how it's loaded, is the focus of this chapter.
- We can use the Jakarta Persistence Query Language (JPQL), a full object- oriented query language based on strings such as `select i from Item i where i.id = ?`.
- The `CriteriaQuery` interface provides a type-safe and object-oriented way to perform queries without string manipulation.
- We can write native SQL queries, call stored procedures, and let Hibernate take care of mapping the JDBC result sets to instances of the domain model classes.

In our JPA applications, we'll use a combination of these techniques. By now you should be familiar with the basic Jakarta Persistence API for retrieving by identifier. We'll keep our JPQL and `CriteriaQuery` exam-

ples as simple as possible, and you won't need the SQL query-mapping features.

Major new features in JPA 2

We can manually check the initialization state of an entity or an entity property with the new `PersistenceUtil` static helper class. We can also create standardized declarative fetch plans with the new `EntityGraph` API.

This chapter analyzes what happens behind the scenes when we navigate the graph of the domain model and Hibernate retrieves data on demand. In all the examples, we'll interpret the SQL executed by Hibernate in a comment right immediately after the operation that triggered the SQL execution.

What Hibernate loads depends on the *fetch plan*: we define the sub-graph of the network of objects that should be loaded. Then we pick the right *fetch strategy*, defining *how* the data should be loaded. We can store the selection of plan and strategy as a *fetch profile* and reuse it.

Defining fetch plans and what data should be loaded by Hibernate relies on two fundamental techniques: *lazy* and *eager* loading of nodes in the network of objects.

12.1 Lazy and eager loading

At some point we must decide what data should be loaded into memory from the database. When we execute `entityManager.find(Item.class, 123)`, what is available in memory and loaded into the persistence context? What happens if we use `EntityManager#getReference()` instead?

In the domain-model mapping, we define the global *default fetch plan*, with the `FetchType.LAZY` and `FetchType.EAGER` options on associations and collections. This plan is the default setting for all operations involving the persistent domain model classes. It's always active when we load an entity instance by identifier and when we navigate the entity graph by following associations and iterating through persistent collections.

Our recommended strategy is a *lazy* default fetch plan for all entities and collections. If we map all of the associations and collections with `FetchType.LAZY`, Hibernate will only load the data we're accessing. As we navigate the graph of the domain model instances, Hibernate will load

data on demand, bit by bit. We can then override this behavior on a per-case basis when necessary.

To implement lazy loading, Hibernate relies on runtime-generated entity placeholders called *proxies* and on *smart wrappers* for collections.

12.1.1 Understanding entity proxies

Consider the `getReference()` method of the `EntityManager` API. In section 10.2.4, we took a first look at this operation and at how it may return a proxy. Let's further explore this important feature and find out how proxies work.

NOTE To execute the examples from the source code, you'll first need to run the Ch12.sql script.

The following code doesn't execute any SQL against the database. All Hibernate does is create an `Item` proxy: it looks (and smells) like the real thing, but it's only a placeholder:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Item item = em.getReference(Item.\1, ITEM_ID);        Ⓐ
assertEquals(ITEM_ID, item.getId());                      Ⓑ
```

Ⓐ There is no database hitting, meaning there is no `SELECT`.

Ⓑ Calling the identifier getter (no field access!) doesn't trigger initialization.

In the persistence context, in memory, we now have this proxy available in persistent state, as shown in figure 12.1.

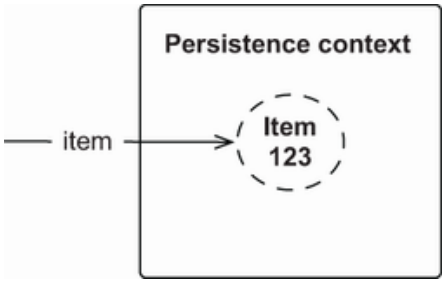

Figure 12.1 The persistence context, under the control of Hibernate, contains an `Item` proxy.

The proxy is an instance of a runtime-generated subclass of `Item`, carrying the identifier value of the entity instance it represents. This is why Hibernate (in line with JPA) requires that entity classes have at least a

public or protected no-argument constructor (the class may have other constructors too). The entity class and its methods must not be final; otherwise, Hibernate can't produce a proxy. Note that the JPA specification doesn't mention proxies; it's up to the JPA provider how lazy loading is implemented.

If we call any method on the proxy that isn't the "identifier getter," we'll trigger initialization of the proxy and hit the database. If we call `item.getName()`, the SQL `SELECT` to load the `Item` will be executed. The previous example called `item.getId()` without triggering initialization because `getId()` *is* the identifier getter method in the given mapping; the `getId()` method was annotated with `@Id`. If `@Id` was on a field, then calling `getId()`, just like calling any other method, would initialize the proxy. (Remember that we usually prefer mappings and access on fields, because this allows more freedom when designing accessor methods; see section 3.2.3. It's up to you whether calling `getId()` without initializing a proxy is more important.)

With proxies, be careful how you compare classes. Because Hibernate generates the proxy class, it has a funny-looking name, and it is *not* equal to `Item.class`:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

assertNotEquals(Item.class, item.getClass());      Ⓐ
assertEquals(
    Item.class,
    HibernateProxyHelper.getClassWithoutInitializingProxy(item)
);
```

Ⓐ The class is runtime generated and named something like `Item$HibernateProxy$BLsrPly8`.

If we really must get the actual type represented by a proxy, we can use the `HibernateProxyHelper`.

JPA provides `PersistenceUtil`, which we can use to check the initialization state of an entity or any of its attributes:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

PersistenceUtil persistenceUtil = Persistence.getPersistenceUtil();
assertFalse(persistenceUtil.isLoaded(item));
assertFalse(persistenceUtil.isLoaded(item, "seller"));
```

```
assertFalse(Hibernate.isInitialized(item));
// assertFalse(Hibernate.isInitialized(item.getSeller()));      Ⓐ
```

Ⓐ Executing this line of code would effectively trigger the initialization of the item.

The `isLoaded()` method also accepts the name of a property of the given entity (proxy) instance, checking its initialization state. Hibernate offers an alternative API with `Hibernate.isInitialized()`. If we call `item.getSeller()`, though, the `item` proxy is initialized first.

Hibernate also offers a utility method for quick-and-dirty initialization of proxies:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Hibernate.initialize(item);                                      Ⓐ
// select * from ITEM where ID = ?
assertFalse(Hibernate.isInitialized(item.getSeller()));          Ⓑ
Hibernate.initialize(item.getSeller());                          Ⓒ
// select * from USERS where ID = ?
```

Ⓐ The first call hits the database and loads the `Item` data, populating the proxy with the item's name, price, and so on.

Ⓑ Make sure the `@ManyToOne` default of `EAGER` has been overridden with `LAZY`. That is why the `seller` of the `item` is not yet initialized.

Ⓒ By initializing the `seller` of the `item`, we hit the database and load the `User` data.

The `seller` of the `Item` is a `@ManyToOne` association mapped with `FetchType.LAZY`, so Hibernate creates a `User` proxy when the `Item` is loaded. We can check the `seller` proxy state and load it manually, just like with the `Item`. Remember that the JPA default for `@ManyToOne` is `FetchType.EAGER`! We usually want to override this to get a lazy default fetch plan, as we demonstrated in section 8.3.1 and again here:

```
Path: Ch12/proxy/src/main/java/com/manning/javapersistence/ch12/proxy
➥ /Item.java

@Entity
public class Item {
    @ManyToOne(fetch = FetchType.LAZY)
    public User getSeller() {
        return seller;
```

```
    }
    //  . . .
}
```

With such a lazy fetch plan, we might run into a `LazyInitializationException`. Consider the following code:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Item item = em.find(Item.class, ITEM_ID);
// select * from ITEM where ID = ?
em.detach(item);
em.detach(item.getSeller());
// em.close();
PersistenceUtil persistenceUtil = Persistence.getPersistenceUtil();
assertTrue(persistenceUtil.isLoaded(item));
assertFalse(persistenceUtil.isLoaded(item, "seller"));
assertEquals(USER_ID, item.getSeller().getId());
//assertNotNull(item.getSeller().getUsername());
```

Ⓐ An `Item` entity instance is loaded in the persistence context. Its `seller` isn't initialized; it's a `User` proxy.

Ⓑ We can manually detach the data from the persistence context or close the persistence context and detach everything.

Ⓒ The `PersistenceUtil` helper works without a persistence context. We can check at any time whether the data we want to access has been loaded.

Ⓓ In detached state, we can call the identifier getter method of the `User` proxy.

Ⓔ Calling any other method on the proxy, such as `getUsername()`, will throw a `LazyInitializationException`. Data can only be loaded on demand while the persistence context manages the proxy, not in detached state.

How does lazy loading of one-to-one associations work?

Lazy loading for one-to-one entity associations is sometimes confusing for new Hibernate users. If we consider one-to-one associations based on shared primary keys (see section 9.1.1), an association can be proxied only if it's `optional=false`. For example, an `Address` always has a reference to a `User`. If this association is nullable and optional, Hibernate

must first hit the database to find out whether it should apply a proxy or a null, and the purpose of lazy loading is to not hit the database at all.

Hibernate proxies are useful beyond simple lazy loading. For example, we can store a new `Bid` without loading any data into memory.

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Item item = em.getReference(Item.class, ITEM_ID);
User user = em.getReference(User.class, USER_ID);
Bid newBid = new Bid(new BigDecimal("99.00"));
newBid.setItem(item);
newBid.setBidder(user);
em.persist(newBid);                                    Ⓐ
// insert into BID values (?, ? ,? ,  . . . )
```

Ⓐ There is no SQL `SELECT` in this procedure, only one `INSERT`.

The first two calls produce proxies of `Item` and `User`, respectively. Then the `item` and `bidder` association properties of the transient `Bid` are set with the proxies. The `persist()` call queues one SQL `INSERT` when the persistence context is flushed, and no `SELECT` is necessary to create the new row in the `BID` table. All key values are available as identifier values of the `Item` and `User` proxy.

Runtime proxy generation, as provided by Hibernate, is an excellent choice for transparent lazy loading. The domain model classes don't have to implement any special type or supertype, as some older ORM solutions require. No code generation or post-processing of bytecode is needed either, simplifying the build procedure. But you should be aware of some potentially negative aspects:

- Some runtime proxies aren't completely transparent, such as polymorphic associations that are tested with `instanceof`. This problem was demonstrated in section 7.8.1.
- With entity proxies, you have to be careful not to access fields directly when writing custom `equals()` and `hashCode()` methods, as discussed in section 10.3.2.

- Proxies can only be used to lazy-load entity associations. They can't be used to lazy-load individual basic properties or embedded components, such as `Item# description` or `User#homeAddress`. If you set the `@Basic(fetch = FetchType .LAZY)` hint on such a property, Hibernate ignores it; the value is eagerly loaded when the owning entity instance is loaded. Optimizing at the level of individual columns selected in SQL is unnecessary if you aren't working with either a significant number of optional or nullable columns, or columns containing large values that have to be retrieved on demand because of the physical limitations of the system. Large values are best represented with large objects (LOBs) instead; they provide lazy loading by definition (see "Binary and large value types" in section 6.3.1).

Proxies enable lazy loading of entity instances. For persistent collections, Hibernate has a slightly different approach.

12.1.2 Lazy persistent collections

We map persistent collections with either `@ElementCollection` for a collection of elements of basic or embeddable types or with `@OneToMany` and `@ManyToMany` for many-valued entity associations. These collections are, unlike `@ManyToOne`, lazy-loaded by default. We don't have to specify the `FetchType.LAZY` option on the mapping.

The lazy `bids` one-to-many collection is only loaded when accessed:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Item item = em.find(Item.class, ITEM_ID);
// select * from ITEM where ID = ?
Set<Bid> bids = item.getBids();
PersistenceUtil persistenceUtil = Persistence.getPersistenceUtil();
assertFalse(persistenceUtil.isLoaded(item, "bids"));
assertTrue(Set.class.isAssignableFrom(bids.getClass()));
assertNotEquals(HashSet.class, bids.getClass());
assertEquals(org.hibernate.collection.internal.PersistentSet.class,
➥ bids.getClass());
```

Ⓐ The `find()` operation loads the `Item` entity instance into the persistence context, as you can see in figure 12.2.

Ⓑ The `Item` instance has a reference to an uninitialized `Set` of `bids`. It also has a reference to an uninitialized `User` proxy: the `seller`.

Ⓒ The `bids` field is a `Set`.

Ⓓ However, the `bids` field is not a `HashSet`.

Ⓔ The `bids` field is a Hibernate proxy class.

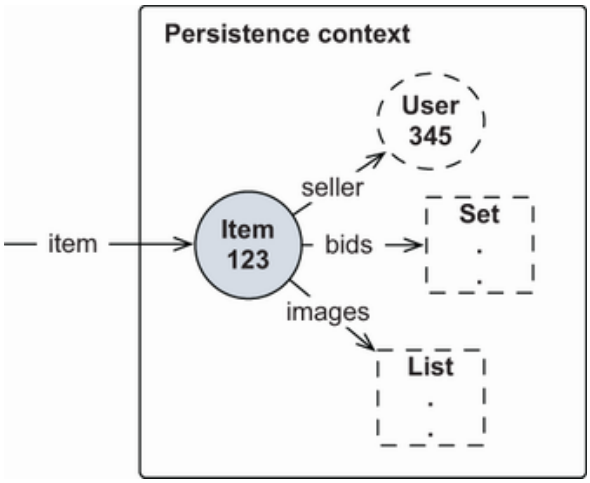

Figure 12.2 Proxies and collection wrappers are the boundary of the loaded graph under Hibernate control.

Hibernate implements lazy loading (and dirty checking) of collections with its own special implementations called *collection wrappers*. Although the `bids` certainly look like a `Set`, Hibernate replaced the implementation with an `org.hibernate.collection.internal.PersistentSet` while we weren't looking. It's not a `HashSet`, but it has the same behavior. That's why it's so important to program with interfaces in the domain model and only rely on `Set` and not `HashSet`. Lists and maps work the same way.

These special collections can detect when we access them, and they load their data at that time. As soon as we start iterating through the `bids`, the collection and all bids made for the item are loaded:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Bid firstBid = bids.iterator().next();
// select * from BID where ITEM_ID = ?
// Alternative: Hibernate.initialize(bids);
```

Alternatively, just as for entity proxies, we can call the static `Hibernate.initialize()` utility method to load a collection. It will be completely loaded; we can't say "only load the first two bids," for example. To do that, we'd have to write a query.

For convenience, so we don't have to write many trivial queries, Hibernate offers a proprietary `LazyCollectionOption.EXTRA` setting on collection mappings:

```
Path: Ch12/proxy/src/main/java/com/manning/javapersistence/ch12/proxy
➥ /Item.java

@Entity
public class Item {
    @OneToMany(mappedBy = "item")
    @org.hibernate.annotations.LazyCollection(
       org.hibernate.annotations.LazyCollectionOption.EXTRA
    )
    public Set<Bid> getBids() {
        return bids;
    }
    //  . . .
}
```

With `LazyCollectionOption.EXTRA`, the collection supports operations that don't trigger initialization. For example, we could ask for the collection's size:

```
Path: Ch12/proxy/src/test/java/com/manning/javapersistence/ch12/proxy
➥ /LazyProxyCollections.java

Item item = em.find(Item.class, ITEM_ID);
// select * from ITEM where ID = ?
assertEquals(3, item.getBids().size());
// select count(b) from BID b where b.ITEM_ID = ?
```

The `size()` operation triggers a `SELECT COUNT()` SQL query but doesn't load the `bids` into memory. On all extra-lazy collections, similar queries are executed for the `isEmpty()` and `contains()` operations. An extra-lazy `Set` checks for duplicates with a simple query when we call `add()`. An extra-lazy `List` only loads one element if we call `get(index)`. For `Map`, extra-lazy operations are `containsKey()` and `containsValue ()`.

12.1.3 Eager loading of associations and collections

We've recommended a lazy default fetch plan, with `FetchType.LAZY` on all the association and collection mappings. Sometimes, although not often, we want the opposite: to specify that a particular entity association or collection should always be loaded. We want the guarantee that this data is available in memory without an additional database hit.

More important, we want a guarantee that, for example, we can access the `seller` of an `Item` once the `Item` instance is in detached state. When the persistence context is closed, lazy loading is no longer available. If `seller` were an uninitialized proxy, we'd get a

`LazyInitializationException` when we accessed it in detached state. For data to be available in detached state, we need to either load it manually while the persistence context is still open or, if we *always* want it loaded, change the fetch plan to be eager instead of lazy.

Let's assume that we always require the `seller` and the `bids` of an `Item` to be loaded:

```
Path: Ch12/eagerjoin/src/main/java/com/manning/javapersistence/ch12
➥ /eagerjoin/Item.java

@Entity
public class Item {
    @ManyToOne(fetch = FetchType.EAGER)                          Ⓐ
    private User seller;
    @OneToMany(mappedBy = "item", fetch = FetchType.EAGER)       Ⓑ
    private Set<Bid> bids = new HashSet<>();
    //  . . .
}
```

Ⓐ `FetchType.EAGER` is the default for entity instances.

Ⓑ Generally, `FetchType.EAGER` on a collection is not recommended. Unlike `FetchType.LAZY`, which is a hint the JPA provider can ignore, `FetchType.EAGER` is a hard requirement. The provider has to guarantee that the data is loaded and available in detached state; it can't ignore the setting.

Consider the collection mapping: is it really a good idea to say "whenever an item is loaded into memory, load the bids of the item right away too"? Even if we only want to display the item's name or find out when the auction ends, all bids will be loaded into memory. Always eager-loading collections, with `FetchType.EAGER` as the default fetch plan in the mapping, usually isn't a great strategy. (Later in this chapter, we'll analyze the *Cartesian product problem*, which appears if we eagerly load several collections.) It's best if we leave collections set with the default `FetchType.LAZY`.

If we now `find()` an `Item` (or force the initialization of an `Item` proxy), both the `seller` and all the `bids` are loaded as persistent instances into the persistence context:

```
Path: Ch12/eagerjoin/src/test/java/com/manning/javapersistence/ch12
➥ /eagerjoin/EagerJoin.java

Item item = em.find(Item.class, ITEM_ID);
// select i.*, u.*, b.*
```

```
//  from ITEM i
//   left outer join USERS u on u.ID = i.SELLER_ID
//   left outer join BID b on b.ITEM_ID = i.ID
//  where i.ID = ?
em.detach(item);                                                    Ⓐ
assertEquals(3, item.getBids().size());                             Ⓑ
assertNotNull(item.getBids().iterator().next().getAmount());
assertEquals("johndoe", item.getSeller().getUsername());            Ⓒ
```

Ⓐ When calling `detach()`, the fetching is done. There will be no more lazy loading.

Ⓑ In detached state, the `bids` collection is available, so we can check its size.

Ⓒ In detached state, the `seller` is available, so we can check its name.

For the `find()`, Hibernate executes a single SQL `SELECT` and `JOIN`s three tables to retrieve the data. You can see the contents of the persistence context in figure 12.3. Note how the boundaries of the loaded graph are represented: each `Bid` has a reference to an uninitialized `User` proxy, the `bidder`. If we now detach the `Item`, we access the loaded `seller` and `bids` without causing a `LazyInitializationException`. If we try to access one of the `bidder` proxies, we'll get an exception.

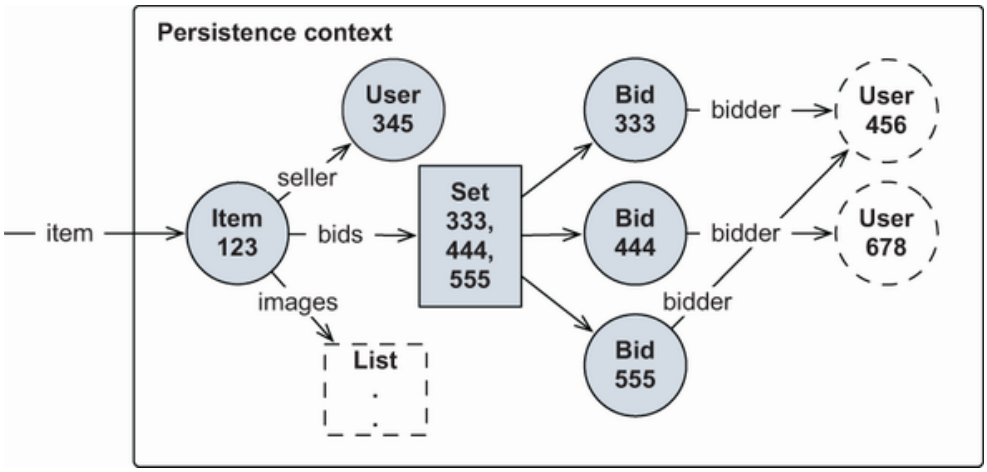

Figure 12.3 The seller and the bids of an `Item` are loaded in the Hibernate persistence context.

Next, we'll investigate *how* data should be loaded when we find an entity instance by identity and when we navigate the network, using the pointers of the mapped associations and collections. We're interested in what SQL is executed and in finding the ideal *fetch strategy*.

In the following examples, we'll assume that the domain model has a lazy default fetch plan. Hibernate will only load the data we explicitly request

and the associations and collections we access.

12.2 Selecting a fetch strategy

Hibernate executes SQL `SELECT` statements to load data into memory. If we load an entity instance, one or more `SELECT` s are executed, depending on the number of tables involved and the *fetching strategy* we've applied. Our goal is to minimize the number of SQL statements and to simplify the SQL statements so that querying can be as efficient as possible.

Consider our recommended fetch plan from earlier in this chapter: every association and collection should be loaded on demand, lazily. This default fetch plan will most likely result in too many SQL statements, each loading only one small piece of data. This will lead to the *n+1 selects problem*, so we'll look at this first. The alternative fetch plan, using eager loading, will result in fewer SQL statements, because larger chunks of data will be loaded into memory with each SQL query. We might then see the *Cartesian product problem*, as SQL result sets become too large.

We need to find a middle ground between these two extremes: the ideal fetching strategy for each procedure and use case in our application. Like with fetch plans, we can set a global fetching strategy in the mappings: a default setting that is always active. Then, for a particular procedure, we might override the default fetching strategy with a custom JPQL `CriteriaQuery` , or even a SQL query.

First, let's investigate the fundamental issues, starting with the *n+1 selects* problem.

12.2.1 The n+1 selects problem

This problem is easy to understand with some example code. Let's assume that we mapped a lazy fetch plan, so everything is loaded on demand. The following code checks whether the `seller` of each `Item` has a `username` :

```
Path: Ch12/nplusoneselects/src/test/java/com/manning/javapersistence/c
➥ /nplusoneselects/NPlusOneSelects.java

List<Item> items =
           em.createQuery("select i from Item i").getResultList();
// select * from ITEM
for (Item item : items) {
    assertNotNull(item.getSeller().getUsername());      Ⓐ
    // select * from USERS where ID = ?
}
```

Ⓐ Whenever we access a seller, each of them must be loaded with an additional `SELECT`.

You can see the one SQL `SELECT` that load the `Item` entity instances. Then, as we iterate through all the `items`, retrieving each `User` requires an additional `SELECT`. This amounts to one query for the `Item` plus *n* queries depending on how many items we have and whether a particular `User` is selling more than one `Item`. Obviously, this is a very inefficient strategy if we know we'll be accessing the `seller` of each `Item`.

We can see the same problem with lazily loaded collections. The following example checks whether each `Item` has any `bids`:

```
Path: Ch12/nplusoneselects/src/test/java/com/manning/javapersistence/c
➥ /nplusoneselects/NPlusOneSelects.java

List<Item> items = em.createQuery("select i from Item i").getResultLis
// select * from ITEM
for (Item item : items) {
    assertTrue(item.getBids().size() > 0);      Ⓐ
    // select * from BID where ITEM_ID = ?
}
```

Ⓐ Each `bids` collection has to be loaded with an additional `SELECT`.

Again, if we know we'll be accessing each `bids` collection, loading only one at a time is inefficient. If we have 100 bids, we'll execute 101 SQL queries!

With what we know so far, we might be tempted to change the default fetch plan in the mappings and put a `FetchType.EAGER` on the `seller` or `bids` associations. But doing so can lead to our next topic: the *Cartesian product* problem.

12.2.2 The Cartesian product problem

If we look at the domain and data model and say, "Every time I need an `Item`, I also need the `seller` of that `Item`," we can map the association with `FetchType.EAGER` instead of a lazy fetch plan. We want a guarantee that whenever an `Item` is loaded, the `seller` will be loaded right away—we want that data to be available when the `Item` is detached and the persistence context is closed:

```
Path: Ch12/cartesianproduct/src/main/java/com/manning/javapersistence/
➥ /cartesianproduct/Item.java
```

```
@Entity
public class Item {
    @ManyToOne(fetch = FetchType.EAGER)
    private User seller;
    //  . . .
}
```

To implement the eager fetch plan, Hibernate uses an SQL `JOIN` operation to load an `Item` and a `User` instance in one `SELECT`:

```
 item = em.find(Item.class, ITEM_ID);
// select i.*, u.*
//  from ITEM i
//   left outer join USERS u on u.ID = i.SELLER_ID
//  where i.ID = ?
```

The result set contains one row with data from the `ITEM` table combined with data from the `USERS` table, as shown in figure 12.4.

i.ID	i.NAME	i.SELLER_ID	...	u.ID	u.USERNAME	...
1	One	2	...	2	johndoe	...

Figure 12.4 Hibernate joins two tables to eagerly fetch associated rows.

Eager fetching with the default `JOIN` strategy isn't problematic for `@ManyToOne` and `@OneToOne` associations. We can eagerly load, with one SQL query and `JOIN`s, an `Item`, its `seller`, the `User`'s `Address`, the `City` they live in, and so on. Even if we map all these associations with `FetchType.EAGER`, the result set will have only one row.

Hibernate has to stop following the `FetchType.EAGER` plan at *some* point. The number of tables joined depends on the global `hibernate.max_fetch_depth` configuration property, and by default, no limit is set. Reasonable values are small, usually from 1 to 5. We can even disable `JOIN` fetching of `@ManyToOne` and `@OneToOne` associations by setting the property to 0. If Hibernate reaches the limit, it will still eagerly load the data according to the fetch plan, but with additional `SELECT` statements. (Note that some database dialects may preset this property; for example, `MySQLDialect` sets it to 2.)

Eagerly loading collections with `JOIN`s, on the other hand, can lead to serious performance concerns. If we also switched to `FetchType.EAGER` for the `bids` and `images` collections, we'd run into the *Cartesian product problem*.

This problem appears when we eagerly load two collections with one SQL query and a `JOIN` operation. First, let's create such a fetch plan and then look at the problem:

```
Path: Ch12/cartesianproduct/src/main/java/com/manning/javapersistence/
➥ /cartesianproduct/Item.java

@Entity
public class Item {
    @OneToMany(mappedBy = "item", fetch = FetchType.EAGER)
    private Set<Bid> bids = new HashSet<>();
    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "IMAGE")
    @Column(name = "FILENAME")
    private Set<String> images = new HashSet<>();
    //  . . .
}
```

It doesn't matter whether both collections are `@OneToMany`, `@ManyToMany`, or `@ElementCollection`. Eager fetching more than one collection at once with the SQL `JOIN` operator is the fundamental problem, no matter what the collection's content is. If we load an `Item`, Hibernate executes the problematic SQL statement:

```
Path: Ch12/cartesianproduct/src/test/java/com/manning/javapersistence/
➥ /cartesianproduct/CartesianProduct.java

Item item = em.find(Item.class, ITEM_ID);
// select i.*, b.*, img.*
//  from ITEM i
//   left outer join BID b on b.ITEM_ID = i.ID
//   left outer join IMAGE img on img.ITEM_ID = i.ID
//  where i.ID = ?
em.detach(item);
assertEquals(3, item.getImages().size();
assertEquals(3, item.getBids().size());
```

As you can see, Hibernate obeyed the eager fetch plan, and we can access the `bids` and `images` collections in detached state. The problem is *how* they were loaded, with an SQL `JOIN` that results in a product. Look at the result set in figure 12.5.

i.ID	i.NAME	...	b.ID	b.AMOUNT	img.FILENAME
1	One	...	1	99.00	foo.jpg
1	One	...	1	99.00	bar.jpg
1	One	...	1	99.00	baz.jpg
1	One	...	2	100.00	foo.jpg
1	One	...	2	100.00	bar.jpg
1	One	...	2	100.00	baz.jp
1	One	...	3	101.00	foo.jpg
1	One	...	3	101.00	bar.jpg
1	One	...	3	101.00	baz.jpg

Figure 12.5 A product is the result of two joins with many rows.

This result set contains many redundant data items, and only the shaded cells are relevant for Hibernate. The `Item` has three bids and three images. The size of the product depends on the size of the collections we're retrieving: 3 × 3 is 9 rows total. Now imagine that we have an `Item` with 50 `bids` and 5 `images` —we'd see a result set with possibly 250 rows! We can create even larger SQL products when we write our own queries with JPQL or `CriteriaQuery` ; imagine what would happen if we load 500 items and eager fetch dozens of bids and images with `JOIN` s.

Considerable processing time and memory are required on the database server to create such results, which then must be transferred across the network. If you're hoping that the JDBC driver will compress the data on the wire somehow, you're probably expecting too much from database vendors. Hibernate immediately removes all duplicates when it marshals the result set into persistent instances and collections; the information in cells that aren't shaded in figure 12.5 will be ignored. Obviously, we can't remove these duplicates at the SQL level; the SQL `DISTINCT` operator doesn't help here.

Instead of using one SQL query with an extremely large result, three separate queries would be faster for retrieving an entity instance and two collections at the same time. We'll focus on this kind of optimization next and see how we can find and implement the best fetch strategy. We'll start again with a default lazy fetch plan and try to solve the *n*+1 selects problem first.

12.2.3 Prefetching data in batches

If Hibernate fetches every entity association and collection only on demand, many additional SQL `SELECT` statements may be needed to complete a particular procedure. As before, consider a routine that checks whether the `seller` of each `Item` has a `username` . With lazy loading,

this would require one `SELECT` to get all `Item` instances and *n* more `SELECT`s to initialize the `seller` proxy of each `Item`.

Hibernate offers algorithms that can prefetch data. The first algorithm we'll look at is *batch fetching*, and it works as follows: if Hibernate must initialize one `User` proxy, it can go ahead and initialize several with the same `SELECT`. In other words, if we already know that there are several `Item` instances in the persistence context and that they all have a proxy applied to their `seller` association, we may as well initialize several proxies instead of just one when we make the round trip to the database.

Let's see how this works. First, enable batch fetching of `User` instances with a proprietary Hibernate annotation:

```
Path: Ch12/batch/src/main/java/com/manning/javapersistence/ch12/batch
➥ /User.java

@Entity
@org.hibernate.annotations.BatchSize(size = 10)
@Table(name = "USERS")
public class User {
    //  . . .
}
```

This setting tells Hibernate that it may load up to 10 `User` proxies if one has to be loaded, all with the same `SELECT`. Batch fetching is often called a *blind-guess optimization* because we don't know how many uninitialized `User` proxies may be in a particular persistence context. We can't say for sure that 10 is an ideal value—it's a guess. We know that instead of *n*+1 SQL queries, we'll now see queries, a significant reduction. Reasonable values are usually small because we don't want to load too much data into memory either, especially if we aren't sure we'll need it.

This is the optimized procedure, which checks the `username` of each `seller`:

```
Path: Ch12/batch/src/test/java/com/manning/javapersistence/ch12/batch
➥ /Batch.java

List<Item> items = em.createQuery("select i from Item i",
➥ Item.class).getResultList();
// select * from ITEM
for (Item item : items) {
    assertNotNull(item.getSeller().getUsername());
    // select * from USERS where ID in (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
}
```

Note the SQL query that Hibernate executes while we iterate through the `items`. When we call `item.getSeller().getUserName()` for the first time, Hibernate must initialize the first `User` proxy. Instead of only loading a single row from the `USERS` table, Hibernate retrieves several rows, and up to 10 `User` instances are loaded. Once we access the eleventh `seller`, another 10 are loaded in one batch, and so on, until the persistence context contains no uninitialized `User` proxies.

What is the real batch-fetching algorithm?

Our explanation of batch loading in section 12.2.3 is somewhat simplified, and you may see a slightly different algorithm in practice.

As an example, imagine a batch size of 32. At startup time, Hibernate creates several batch loaders internally. Each loader knows how many proxies it can initialize. The goal is to minimize the memory consumption for loader creation and to create enough loaders that every possible batch fetch can be produced. Another goal is obviously to minimize the number of SQL queries. To initialize 31 proxies, Hibernate executes 3 batches (you probably expected 1, because 32 > 31). The batch loaders that are applied are 16, 10, and 5, as automatically selected by Hibernate.

You can customize this batch-fetching algorithm with the `hibernate.batch_fetch_style` property in the persistence unit configuration. The default is `LEGACY`, which builds and selects several batch loaders on startup. Other options are `PADDED` and `DYNAMIC`. With `PADDED`, Hibernate builds only one batch loader SQL query on startup with placeholders for 32 arguments in the `IN` clause and then repeats bound identifiers if fewer than 32 proxies have to be loaded. With `DYNAMIC`, Hibernate dynamically builds the batch SQL statement at runtime, when it knows the number of proxies to initialize.

Batch fetching is also available for collections:

```
Path: Ch12/batch/src/main/java/com/manning/javapersistence/ch12/batch
➥ /Item.java

@Entity
public class Item {
    @OneToMany(mappedBy = "item")
    @org.hibernate.annotations.BatchSize(size = 5)
    private Set<Bid> bids = new HashSet<>();
    //  . . .
}
```

If we now force the initialization of one `bids` collection, up to five more `Item#bids` collections, if they're uninitialized in the current persistence context, are loaded right away:

```
Path: Ch12/batch/src/test/java/com/manning/javapersistence/ch12/batch
➥ /Batch.java

List<Item> items = em.createQuery("select i from Item i",
➥ Item.class).getResultList();
// select * from ITEM
for (Item item : items) {
    assertTrue(item.getBids().size() > 0);
    // select * from BID where ITEM_ID in (?, ?, ?, ?, ?)
}
```

When we call `item.getBids().size()` for the first time while iterating, a whole batch of `Bid` collections are preloaded for the other `Item` instances.

Batch fetching is a simple and often smart optimization that can significantly reduce the number of SQL statements that would otherwise be necessary to initialize all the proxies and collections. Although we may prefetch data we won't need, and consume more memory, the reduction in database round trips can make a huge difference. Memory is cheap, but scaling database servers isn't.

Another prefetching algorithm that isn't a blind guess uses subselects to initialize many collections with a single statement.

12.2.4 Prefetching collections with subselects

A potentially better strategy for loading all the `bids` of several `Item` instances is prefetching with a subselect. To enable this optimization, add a Fetch Hibernate annotation to the collection mapping, having the SUBSELECT parameter:

```
Path: Ch12/subselect/src/main/java/com/manning/javapersistence/ch12
➥ /subselect/Item.java

@Entity
public class Item {
    @OneToMany(mappedBy = "item")
    @org.hibernate.annotations.Fetch(
       org.hibernate.annotations.FetchMode.SUBSELECT
    )
    private Set<Bid> bids = new HashSet<>();
```

```
    //  . . .
}
```

Hibernate now initializes all `bids` collections for all loaded `Item` instances as soon as we force the initialization of one `bids` collection:

```
Path: Ch12/subselect/src/test/java/com/manning/javapersistence/ch12
➥ /subselect/Subselect.java

List<Item> items = em.createQuery("select i from Item i",
➥ Item.class).getResultList();
// select * from ITEM
for (Item item : items) {
    assertTrue(item.getBids().size() > 0);
    // select * from BID where ITEM_ID in (
    //  select ID from ITEM
    // )
}
```

Hibernate remembers the original query used to load the `items`. It then embeds this initial query (slightly modified) in a subselect, retrieving the collection of `bids` for each `Item`.

Note that the original query that is rerun as a subselect is only remembered by Hibernate for a particular persistence context. If we detach an `Item` instance without initializing the collection of `bids`, and then merge it with a new persistence context and start iterating through the collection, no prefetching of other collections occurs.

Batch and subselect prefetching reduce the number of queries necessary for a particular procedure if you stick with a global lazy fetch plan in the mappings, helping mitigate the *n*+1 selects problem. If, instead, your global fetch plan has eager loaded associations and collections, you'll have to avoid the Cartesian product problem—for example, by breaking down a `JOIN` query into several `SELECT`s.

12.2.5 Eager fetching with multiple SELECTs

When trying to fetch several collections with one SQL query and `JOIN`s, we'll run into the Cartesian product problem, as discussed earlier. Instead of a `JOIN` operation, we can tell Hibernate to eagerly load data with additional `SELECT` queries and hence avoid large results and SQL products with duplicates:

```
Path: Ch12/eagerselect/src/main/java/com/manning/javapersistence/ch12
➥ /eagerselect/Item.java
```

```
@Entity
public class Item {
    @ManyToOne(fetch = FetchType.EAGER)
    @org.hibernate.annotations.Fetch(
       org.hibernate.annotations.FetchMode.SELECT                    Ⓐ
    )
    private User seller;
    @OneToMany(mappedBy = "item", fetch = FetchType.EAGER)
    @org.hibernate.annotations.Fetch(
       org.hibernate.annotations.FetchMode.SELECT                    Ⓐ
    )
    private Set<Bid> bids = new HashSet<>();
    //  . . .
}
```

Ⓐ `FetchMode.SELECT` means that the property should be loaded lazily. The default value is `FetchMode.JOIN`, meaning the property would be retrieved eagerly, via a `JOIN`.

Now when an `Item` is loaded, the `seller` and `bids` have to be loaded as well:

```
Path: Ch12/eagerselect/src/test/java/com/manning/javapersistence/ch12
➥ /eagerselect/EagerSelect.java

Item item = em.find(Item.class, ITEM_ID);                            Ⓐ
// select * from ITEM where ID = ?
// select * from USERS where ID = ?
// select * from BID where ITEM_ID = ?
em.detach(item);
assertEquals(3, item.getBids().size());                              Ⓑ
assertNotNull(item.getBids().iterator().next().getAmount());        Ⓑ
assertEquals("johndoe", item.getSeller().getUsername());            Ⓑ
```

Ⓐ Hibernate uses one `SELECT` to load a row from the `ITEM` table. It then immediately executes two more `SELECT`s: one loading a row from the `USERS` table (the `seller`) and the other loading several rows from the `BID` table (the `bids`). The additional `SELECT` queries aren't executed lazily; the `find()` method produces several SQL queries.

Ⓑ Hibernate followed the eager fetch plan; all data is available in detached state.

Still, all of these settings are global; they're always active. The danger is that adjusting one setting for one problematic case in the application might have negative side effects on some other procedure. Maintaining

this balance can be difficult, so our recommendation is to map every entity association and collection as `FetchType.LAZY`, as mentioned before.

A better approach is to *dynamically* use eager fetching and `JOIN` operations only when needed, for a particular procedure.

12.2.6 Dynamic eager fetching

As in the previous sections, let's say we have to check the `username` of each `Item#seller`. With a lazy global fetch plan, we can load the needed data for this procedure and apply a dynamic eager fetch strategy in a query:

```
Path: Ch12/eagerselect/src/test/java/com/manning/javapersistence/ch12
➥ /eagerselect/EagerQueryUsers.java

List<Item> items =
    em.createQuery("select i from Item i join fetch i.seller", Item.cl
        .getResultList();                                              Ⓐ
// select i.*, u.*
//  from ITEM i
//   inner join USERS u on u.ID = i.SELLER_ID
//  where i.ID = ?
em.close();                                                            Ⓑ
for (Item item : items) {
    assertNotNull(item.getSeller().getUsername());                     Ⓒ
}
```

Ⓐ Apply a dynamic eager strategy in a query.

Ⓑ Detach all.

Ⓒ Hibernate followed the eager fetch plan; all data is available in detached state.

The important keywords in this JPQL query are `join fetch`, telling Hibernate to use a SQL `JOIN` (an `INNER JOIN`, actually) to retrieve the `seller` of each `Item` in the same query. The same query can be expressed with the `CriteriaQuery` API instead of a JPQL string:

```
Path: Ch12/eagerselect/src/test/java/com/manning/javapersistence/ch12
➥ /eagerselect/EagerQueryUsers.java

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Item> criteria = cb.createQuery(Item.class);
Root<Item> i = criteria.from(Item.class);
i.fetch("seller");
criteria.select(i);
```

```
List<Item> items = em.createQuery(criteria).getResultList();        Ⓐ
em.close();                                                         Ⓑ
for (Item item : items) {
    assertNotNull(item.getSeller().getUsername());                  Ⓒ
}
```

Ⓐ Apply an eager strategy in a query dynamically constructed with the `CriteriaQuery` API.

Ⓑ Detach all.

Ⓒ Hibernate followed the eager fetch plan; all data is available in detached state.

Dynamic eager join fetching also works for collections. Here we load all `bids` of each `Item`:

```
Path: Ch12/eagerselect/src/test/java/com/manning/javapersistence/ch12
➥ /eagerselect/EagerQueryBids.java

List<Item> items =
    em.createQuery("select i from Item i left join fetch i.bids",
    ➥ Item.class)
        .getResultList();                                           Ⓐ
// select i.*, b.*
//  from ITEM i
//   left outer join BID b on b.ITEM_ID = i.ID
//  where i.ID = ?
em.close();                                                         Ⓑ
for (Item item : items) {
    assertTrue(item.getBids().size() > 0);                          Ⓒ
}
```

Ⓐ Apply a dynamic eager strategy in a query.

Ⓑ Detach all.

Ⓒ Hibernate followed the eager fetch plan; all data is available in detached state.

Now let's do the same with the `CriteriaQuery` API:

```
Path: Ch12/eagerselect/src/test/java/com/manning/javapersistence/ch12
➥ /eagerselect/EagerQueryBids.java

CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Item> criteria = cb.createQuery(Item.class);
```

```
Root<Item> i = criteria.from(Item.class);
i.fetch("bids", JoinType.LEFT);
criteria.select(i);
List<Item> items = em.createQuery(criteria).getResultList();      Ⓐ
em.close();                                                       Ⓑ
for (Item item : items) {
    assertTrue(item.getBids().size() > 0);                        Ⓒ
}
```

Ⓐ Apply an eager strategy in a query dynamically constructed with the `CriteriaQuery` API.

Ⓑ Detach all.

Ⓒ Hibernate followed the eager fetch plan; all data is available in detached state.

Note that for collection fetching, a `LEFT OUTER JOIN` is necessary, because we also want rows from the `ITEM` table if there are no `bids`.

Writing queries by hand isn't the only available option if we want to override the global fetch plan of the domain model dynamically. We can write *fetch profiles* declaratively.

12.3 Using fetch profiles

Fetch profiles complement the fetching options in the query languages and APIs. They allow us to maintain the profile definitions in either XML or annotation metadata. Early Hibernate versions didn't have support for special fetch profiles, but today Hibernate supports the following:

- *Fetch profiles*—A proprietary API based on the declaration of the profile with `@org.hibernate.annotations.FetchProfile` and execution with `Session #enableFetchProfile()`. This simple mechanism currently supports overriding lazy-mapped entity associations and collections selectively, enabling a `JOIN` eager fetching strategy for a particular unit of work.
- *Entity graphs*—Specified in JPA 2.1, we can declare a graph of entity attributes and associations with the `@EntityGraph` annotation. This fetch plan, or a combination of plans, can be enabled as a hint when executing `EntityManager #find()` or queries (JPQL, criteria). The provided graph controls *what* should be loaded; unfortunately, it doesn't control *how* it should be loaded.

It's fair to say that there is room for improvement here, and we expect future versions of Hibernate and JPA to offer a unified and more powerful API.

We can externalize JPQL and SQL statements and move them to metadata. A JPQL query *is* a declarative (named) fetch profile; what we're missing is the ability to overlay different plans easily on the same base query. We've seen some creative solutions with string manipulation that are best avoided. With criteria queries, on the other hand, we already have the full power of Java available to organize the query-building code. The value of entity graphs is being able to reuse fetch plans across any kind of query.

Let's talk about Hibernate fetch profiles first and how we can override a global lazy fetch plan for a particular unit of work.

12.3.1 Declaring Hibernate fetch profiles

Hibernate fetch profiles are global metadata; they're declared for the entire persistence unit. Although we could place the `@FetchProfile` annotation on a class, we prefer it as package-level metadata in a package-info.java file:

```
Path: Ch12/profile/src/main/java/com/manning/javapersistence/ch12/prof
➥ /package-info.java

@org.hibernate.annotations.FetchProfiles({
    @FetchProfile(name = Item.PROFILE_JOIN_SELLER,          Ⓐ
        fetchOverrides = @FetchProfile.FetchOverride(       Ⓑ
            entity = Item.class,
            association = "seller",
            mode = FetchMode.JOIN                           Ⓒ
        )),
    @FetchProfile(name = Item.PROFILE_JOIN_BIDS,
        fetchOverrides = @FetchProfile.FetchOverride(
            entity = Item.class,
            association = "bids",
            mode = FetchMode.JOIN
        ))
})
```

Ⓐ Each profile has a name. This is a simple string isolated in a constant.

Ⓑ Each override in a profile names one entity association or collection.

Ⓒ `FetchMode.JOIN` means the property will be retrieved eagerly, via a `JOIN`.

The profiles can now be enabled for a unit of work. We need the Hibernate API to enable a profile. It's then active for any operation in that unit of work. The `Item#seller` may be fetched with a join in the same SQL statement whenever an `Item` is loaded with this `EntityManager`.

We can overlay another profile on the same unit of work. In the following example, the `Item#seller` and the `Item#bids` collections will be fetched with a join in the same SQL statement whenever an `Item` is loaded.

```
Path: Ch12/profile/src/test/java/com/manning/javapersistence/ch12/prof
➥ /Profile.java

Item item = em.find(Item.class, ITEM_ID);
em.clear();
em.unwrap(Session.class).enableFetchProfile(Item.PROFILE_JOIN_SELLER);
item = em.find(Item.class, ITEM_ID);
em.clear();
em.unwrap(Session.class).enableFetchProfile(Item.PROFILE_JOIN_BIDS);
item = em.find(Item.class, ITEM_ID);
```

Ⓐ The `Item#seller` is mapped lazily, so the default fetch plan only retrieves the `Item` instance.

Ⓑ Fetch the `Item#seller` with a join in the same SQL statement whenever an `Item` is loaded with this `EntityManager`.

Ⓒ Fetch `Item#seller` and `Item#bids` with a join in the same SQL statement whenever an `Item` is loaded.

Basic Hibernate fetch profiles can be an easy solution for fetching optimization in smaller or simpler applications. Starting with JPA 2.1, the introduction of *entity graphs* enables similar functionality in a standard fashion.

12.3.2 Working with entity graphs

An entity graph is a declaration of entity nodes and attributes, overriding or augmenting the default fetch plan when we execute an `EntityManager#find()` or put a hint on query operations. This is an example of a retrieval operation using an entity graph:

```
Path: Ch12/fetchloadgraph/src/test/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/FetchLoadGraph.java

Map<String, Object> properties = new HashMap<>();
properties.put(
    "javax.persistence.loadgraph",
    em.getEntityGraph(Item.class.getSimpleName())      Ⓐ
);
Item item = em.find(Item.class, ITEM_ID, properties);
// select * from ITEM where ID = ?
```

Ⓐ The name of the entity graph we're using is `Item`, and the hint for the `find()` operation indicates it should be the load graph. This means attributes that are specified by attribute nodes of the entity graph are treated as `FetchType.EAGER`, and attributes that aren't specified are treated according to their specified or default `FetchType` in the mapping.

The following code shows the declaration of this graph and the default fetch plan of the entity class:

```
Path: Ch12/fetchloadgraph/src/main/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/Item.java

@NamedEntityGraphs({
    @NamedEntityGraph          Ⓐ
})
@Entity
public class Item {
    @NotNull
    @ManyToOne(fetch = FetchType.LAZY)
    private User seller;
    @OneToMany(mappedBy = "item")
    private Set<Bid> bids = new HashSet<>();
    @ElementCollection
    private Set<String> images = new HashSet<>();
    //  . . .
}
```

Ⓐ Entity graphs in metadata have names and are associated with an entity class; they're usually declared in annotations at the top of an entity class. We can alternatively put them in XML if we like. If we don't give an entity graph a name, it gets the simple name of its owning entity class, which here is `Item`.

If we don't specify any attribute nodes in the graph, like the empty entity graph in the preceding example, the defaults of the entity class are used. In `Item`, all associations and collections are mapped lazily; this is the default fetch plan. Hence, what we've done so far makes little difference, and the `find()` operation without any hints will produce the same result: the `Item` instance is loaded, and the `seller`, `bids`, and `images` aren't.

Alternatively, we can build an entity graph with an API:

```
Path: Ch12/fetchloadgraph/src/test/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/FetchLoadGraph.java

EntityGraph<Item> itemGraph = em.createEntityGraph(Item.class);
Map<String, Object> properties = new HashMap<>();
```

```
properties.put("javax.persistence.loadgraph", itemGraph);
Item item = em.find(Item.class, ITEM_ID, properties);
```

This is again an empty entity graph with no attribute nodes, given directly to a retrieval operation.

Let's say we want to write an entity graph that changes the lazy default of `Item#seller` to eager fetching when it's enabled:

```
Path: Ch12/fetchloadgraph/src/main/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/Item.java

@NamedEntityGraphs({
    @NamedEntityGraph(
        name = "ItemSeller",
        attributeNodes = {
            @NamedAttributeNode("seller")
        }
    )
})
@Entity
public class Item {
    //  . . .
}
```

We can now enable this graph by name when we want the `Item` and the `seller` eagerly loaded:

```
Path: Ch12/fetchloadgraph/src/test/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/FetchLoadGraph.java

Map<String, Object> properties = new HashMap<>();
properties.put(
    "javax.persistence.loadgraph",
    em.getEntityGraph("ItemSeller")
);
Item item = em.find(Item.class, ITEM_ID, properties);
// select i.*, u.*
//  from ITEM i
//   inner join USERS u on u.ID = i.SELLER_ID
// where i.ID = ?
```

If we don't want to hardcode the graph in annotations, we can build it with the API instead:

```
Path: Ch12/fetchloadgraph/src/test/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/FetchLoadGraph.java
```

```
EntityGraph<Item> itemGraph = em.createEntityGraph(Item.class);
itemGraph.addAttributeNodes(Item_.seller);                              Ⓐ
Map<String, Object> properties = new HashMap<>();
properties.put("javax.persistence.loadgraph", itemGraph);
Item item = em.find(Item.class, ITEM_ID, properties);
// select i.*, u.*
//  from ITEM i
//   inner join USERS u on u.ID = i.SELLER_ID
// where i.ID = ?
```

Ⓐ The `Item_` class belongs to the static metamodel. It is automatically generated by including the Hibernate JPA2 Metamodel Generator dependency in the project. Take a look back at section 3.3.4 for more details.

So far we've seen only properties for the `find()` operation. Entity graphs can also be enabled for queries, as hints:

```
Path: Ch12/fetchloadgraph/src/test/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/FetchLoadGraph.java

List<Item> items =
    em.createQuery("select i from Item i", Item.class)
        .setHint("javax.persistence.loadgraph", itemGraph)
        .getResultList();
// select i.*, u.*
//  from ITEM i
//   left outer join USERS u on u.ID = i.SELLER_ID
```

Entity graphs can be complex. The following declaration shows how to work with reusable subgraph declarations:

```
Path: Ch12/fetchloadgraph/src/main/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/Bid.java

@NamedEntityGraphs({
    @NamedEntityGraph(
        name = "BidBidderItemSellerBids",
        attributeNodes = {
            @NamedAttributeNode(value = "bidder"),
            @NamedAttributeNode(
                value = "item",
                subgraph = "ItemSellerBids"
            )
        },
        subgraphs = {
            @NamedSubgraph(
                name = "ItemSellerBids",
                attributeNodes = {
```

```
                    @NamedAttributeNode("seller"),
                    @NamedAttributeNode("bids")
                })
        }
    )
})
@Entity
public class Bid {
    //  . . .
}
```

This entity graph, when enabled as a load graph when retrieving `Bid` instances, also triggers eager fetching of `Bid#bidder`, the `Bid#item`, and furthermore the `Item#seller` and all `Item#bids`. Although you're free to name your entity graphs any way you like, we recommend that you develop a convention that everyone in your team can follow, and move the strings to shared constants.

With the entity graph API, the previous plan looks like this:

```
Path: Ch12/fetchloadgraph/src/test/java/com/manning/javapersistence/ch
➥ /fetchloadgraph/FetchLoadGraph.java

EntityGraph<Bid> bidGraph = em.createEntityGraph(Bid.class);
bidGraph.addAttributeNodes(Bid_.bidder, Bid_.item);
Subgraph<Item> itemGraph = bidGraph.addSubgraph(Bid_.item);
itemGraph.addAttributeNodes(Item_.seller, Item_.bids);
Map<String, Object> properties = new HashMap<>();
properties.put("javax.persistence.loadgraph", bidGraph);
Bid bid = em.find(Bid.class, BID_ID, properties);
```

We've only seen entity graphs as *load graphs* so far. There is another option: we can enable an entity graph as a *fetch graph* with the `javax.persistence.fetchgraph` hint. If we execute a `find()` or query operation with a fetch graph, any attributes and collections not in the plan will be made `FetchType.LAZY`, and any nodes in the plan will be `FetchType.EAGER`. This effectively ignores all `FetchType` settings in the entity attribute and collection mappings.

Two weak points of the JPA entity graph operations are worth mentioning, because you'll run into them quickly. First, you can only modify fetch plans, not the Hibernate fetch strategy (batch/subselect/join/select). Second, declaring an entity graph in annotations or XML isn't fully type-safe: the attribute names are strings. The `EntityGraph` API at least is type-safe.

Summary

- A fetch profile combines a fetch plan (identifying what data should be loaded) with a fetch strategy (how the data should be loaded), encapsulated in reusable metadata or code.
- You can create a global fetch plan and define which associations and collections should be loaded into memory at all times.
- You can define the fetch plan based on use cases, how to access associated entities and iterate through collections in the application, and which data should be available in detached state.
- You can select the right fetching strategy for the fetch plan. The goal is to minimize the number of SQL statements and the complexity of each SQL statement that must be executed.
- You can use fetching strategies especially to avoid the *n*+1 selects and Cartesian product problems.